Aim: W R a program in C++ to DFraw a line ising DDA alogithm(Straight line)

```
#include<iostream.h>

#include<graphics.h>

#include<conio.h>

#include<math.h>


void main()

{

   int gd, gm;

   float x1, y1, x2, y2, x, y, xdiff, ydiff, steps, xinc, yinc;

   int i;


   clrscr();


   cout<<"Enter the initial coordinates of line: ";

   cin>>x1>>y1;


   cout<<"Enter the terminating coordinates of line: ";

   cin>>x2>>y2;


   xdiff = x2 - x1;

   ydiff = y2 - y1;


   if(abs(xdiff) > abs(ydiff))

      steps = abs(xdiff);

   else

      steps = abs(ydiff);


   xinc = xdiff / steps;

   yinc = ydiff / steps;
```

```
   gd = DETECT;
   initgraph(&gd, &gm, "C:\\TURBOC3\\BGI");

   x = x1;
   y = y1;

   for(i = 0; i <= steps; i++)
   {
      putpixel((int)x, (int)y, WHITE);
      x = x + xinc;
      y = y + yinc;
   }

   getch();
   closegraph();
}
```

Output:

x1 = 100, y1 = 100

x2 = 100, y2 = 200

Aim: W R a program to demonstrate the boundary fill algorithm.

```
#include<stdio.h>
#include<graphics.h>
#include<conio.h>
#include<dos.h>

void boundaryfill(int x, int y, int fcolor, int bcolor)
{
   if(getpixel(x,y) != bcolor && getpixel(x,y) != fcolor)
   {
```

```
        putpixel(x,y,fcolor);

        boundaryfill(x+1,y,fcolor,bcolor);
        boundaryfill(x-1,y,fcolor,bcolor);
        boundaryfill(x,y+1,fcolor,bcolor);
        boundaryfill(x,y-1,fcolor,bcolor);
    }
}

void main()
{
    int gd = DETECT, gm;
    int x, y, radius;

    clrscr();

    printf("Enter the x and y position for circle: ");
    scanf("%d %d",&x,&y);

    printf("Enter the radius of circle: ");
    scanf("%d",&radius);

    initgraph(&gd,&gm,"C:\\TURBOC3\\BGI");

    circle(x,y,radius);

    boundaryfill(x,y,4,15);

    getch();
    closegraph();
}
```

Output:

Enter the x and y position for circle: 200 200

Enter the radius of circle: 100

Aim

Write a program in C++ to implement Bresenham's Line Drawing Algorithm.

```
#include <iostream.h>

#include <conio.h>

#include <graphics.h>

#include <math.h>


void main()

{

   int x1, y1, x2, y2, dx, dy, p, x, y;

   int gd = DETECT, gm;


   clrscr();


   cout<<"Enter the end points of a line:";

   cout<<"\nx1 = ";

   cin>>x1;

   cout<<"y1 = ";

   cin>>y1;

   cout<<"x2 = ";

   cin>>x2;

   cout<<"y2 = ";

   cin>>y2;


   initgraph(&gd,&gm,"C:\\TURBOC3\\BGI");
```

```
    dx = abs(x2 - x1);
    dy = abs(y2 - y1);

    p = 2*dy - dx;

    x = x1;
    y = y1;

    putpixel(x,y,15);

    for(int i=1; i<=dx; i++)
    {
        if(p < 0)
        {
            x = x + 1;
            p = p + 2*dy;
        }
        else
        {
            x = x + 1;
            y = y + 1;
            p = p + 2*(dy - dx);
        }

        putpixel(x,y,15);
    }

    getch();
    closegraph();
}
```

Output:

Enter the end points of a line:

x1 = 100

y1 = 100

x2 = 300

y2 = 300

Modul 2:

Aim

Write a Python program to demonstrate the collision effect of bouncing balls.

Program

```
import pygame
import random
import sys

class Ball:
    def __init__(self, x, y):
        self.velocity = [1, 1]
        self.ball_image = pygame.image.load("beach.png").convert()
        self.ball_rect = self.ball_image.get_rect(center=(x, y))

# Main program
if __name__ == "__main__":

    width = 800
    height = 600
    background_color = (0, 0, 0)

    pygame.init()
    window = pygame.display.set_mode((width, height))
    pygame.display.set_caption("Bouncing Ball Animation")

    num_balls = 3
    ball_list = []

    for number in range(num_balls):
        ball = Ball(random.randint(0, width-50),
                    random.randint(0, height-50))
        ball_list.append(ball)

    while True:

        for event in pygame.event.get():
```

```
        if event.type == pygame.QUIT:
            pygame.quit()
            sys.exit()

    window.fill(background_color)

    for ball in ball_list:

        if ball.ball_rect.left < 0 or ball.ball_rect.right > width:
            ball.velocity[0] = -ball.velocity[0]

        if ball.ball_rect.top < 0 or ball.ball_rect.bottom > height:
            ball.velocity[1] = -ball.velocity[1]

        ball.ball_rect = ball.ball_rect.move(ball.velocity)

        window.blit(ball.ball_image, ball.ball_rect)

    pygame.display.flip()
```

Aim

Write a Python program to flip the display screen with games.

Program

```
import pygame
import sys

pygame.init()

SCREEN_WIDTH = 800
SCREEN_HEIGHT = 600

screen = pygame.display.set_mode((SCREEN_WIDTH, SCREEN_HEIGHT))
pygame.display.set_caption("My Pygame Frame")

clock = pygame.time.Clock()
FPS = 10

running = True

while running:

    for event in pygame.event.get():
        if event.type == pygame.QUIT:
```

```
        running = False

    screen.fill((255, 0, 255))

    pygame.display.flip()

    clock.tick(FPS)

pygame.quit()
sys.exit()
```

Aim: 2D Transformation program (Translation, Scaling, Rotation) using Python, NumPy, and Matplotlib

```
import numpy as np

import matplotlib.pyplot as plt


# Define square coordinates

square = np.array([

    [1, 1],

    [4, 1],

    [4, 4],

    [1, 4],

    [1, 1]

])


# Function to plot shape

def plot_shape(shape, title, color):

    plt.plot(shape[:,0], shape[:,1], color=color, marker='o')

    plt.title(title)

    plt.grid(True)

    plt.axis('equal')


# Create figure

plt.figure(figsize=(12,4))


# 1. Original Object
```

```
plt.subplot(1,4,1)
plot_shape(square, "Original Object", "black")

# 2. Translation
tx, ty = 2, 3
translated = square + np.array([tx, ty])

plt.subplot(1,4,2)
plot_shape(translated, "Translated Object", "blue")

# 3. Scaling
sx, sy = 1.5, 0.5
scaled = square * np.array([sx, sy])

plt.subplot(1,4,3)
plot_shape(scaled, "Scaled Object", "green")

# 4. Rotation
angle = np.radians(45)

rotation_matrix = np.array([
    [np.cos(angle), -np.sin(angle)],
    [np.sin(angle),  np.cos(angle)]
])

rotated = square.dot(rotation_matrix)

plt.subplot(1,4,4)
plot_shape(rotated, "Rotated Object", "red")

plt.show()
```